GESTIÓN DE LA CALIDAD DEL SOFTWARE · UFG

Monedero doc

Monedero electrónico prepago de circuito cerrado — prototipo de demostración

Este documento resume qué es el proyecto, cómo está construido, cómo funciona el backend y cómo se usa la app en cada rol. Está dividido en tres partes para tres expositores (unos 3 minutos cada una) y termina con la demostración y los próximos pasos.

Avance al 25 de octubre de 2026: fases 1 a 6 terminadas. La app ya funciona de punta a punta contra la API real, en web y en Android, y el ingeniero puede probarla desde su propio celular o computadora con un link.

Parte	Tema
1	El proyecto, el stack y cómo se trabajó
2	El backend: API, reglas del dinero, seguridad y pruebas
3	La app en los tres roles, la demostración y los próximos pasos

Repositorio: github.com/dn223/Monedero · El link de la demo se genera el día de la exposición con `dart run tool/share.dart`.

Parte 1 · El proyecto, el stack y cómo se trabajó

Expositor 1 · ≈ 3 minutos

1.1 Qué es Monedero

Una tarjeta prepago como la de un supermercado: se **recarga en caja** y se **gasta solo dentro del comercio** (circuito cerrado). Cada tarjeta tiene un número único de 16 dígitos y un saldo. Aunque es académico, se diseñó **como si manejara dinero real**: no se puede perder ni duplicar un centavo.

1.2 Usuarios

- **Titular:** ve su saldo y su historial, muestra su tarjeta virtual con QR, **paga desde el celular** escribiendo el código de la caja, reporta la pérdida y cambia su PIN.
- **Caja:** lee la tarjeta, cobra (con PIN si supera $25), recarga en múltiplos de $5 y entrega un comprobante.
- **Administración:** ve el tablero con el saldo en circulación, emite y repone tarjetas, autoriza las cajas y revisa la conciliación.

1.3 Stack

Parte	Tecnología	Para qué
App	Flutter (Dart), Material 3, Riverpod, go_router	Una sola app: APK de Android y web instalable
API	Dart con shelf	Aplica las reglas y la seguridad; también sirve la app web
Base de datos	PostgreSQL 17	Única fuente de verdad del saldo
Código compartido	Paquete *core* en Dart	App y API validan igual
Entorno local	Docker Compose y Mailpit	Todo corre en la laptop con un comando, a costo $0
Exposición por link **nuevo**	Túnel de Cloudflare dentro de Docker	Un link *https* temporal, gratis y sin cuentas, para probar desde cualquier lugar
Producción (diseñada)	VPS Contabo, Caddy, pgBackRest a Backblaze B2	HTTPS y respaldos continuos
Tarjetas	QR simulado hoy · NFC MIFARE Classic después	Leer número, UID y firma

1.4 Arquitectura



La app nunca decide el saldo: envía cada operación a la API, que revisa las reglas, y PostgreSQL guarda el resultado y se protege a sí misma. Para la exposición, la misma laptop abre un túnel: el ingeniero entra por el link y la app y la API llegan por el mismo origen.

1.5 Cómo se trabajó

- **Monorepo** con la app, la API, el paquete compartido, la base de datos y la documentación en un solo repositorio de **GitHub**.
- Trabajo por **fases** con entregables verificables; cada fase termina con pruebas en verde, documentación y commit.
- Ramas y Conventional Commits, integración continua (GitHub Actions corre análisis y pruebas en cada cambio).
- **ADR** (registro de decisiones técnicas): 13 hasta hoy; por ejemplo, PostgreSQL en lugar de Firestore, montos en centavos, renovación de sesión y exposición por link.
- **AGENTS.md**: instrucciones únicas para que cada integrante trabaje con su propia IA sin romper las reglas; y una bitácora de sesiones.

Parte 2 · El backend: API, reglas del dinero, seguridad y pruebas

Expositor 2 · ≈ 3 minutos

2.1 Reglas del dinero

- Montos en **centavos enteros** (BIGINT / int), nunca decimales: evita errores de redondeo.
- Recargas solo en **múltiplos de $5**; cobros mayores que cero y **sin sobregiro**; **PIN** si el cobro supera $25.
- Solo una tarjeta **ACTIVE** puede recibir recargas o cobros (estados: PENDING, ACTIVE, BLOCKED, REPLACED).
- Cada tarjeta lleva un **UID** y una **firma HMAC** que solo el servidor puede generar: nadie puede fabricar una tarjeta válida.

2.2 Cómo funciona un cobro

La caja envía el cobro con una **clave de idempotencia**. La API valida el rol, la caja autorizada, la firma de la tarjeta y el monto. Dentro de una sola **transacción**, bloquea la tarjeta (*SELECT … FOR UPDATE*), revisa el saldo, lo actualiza y registra el movimiento. Si la red falla y la caja reintenta con la misma clave, recibe el mismo comprobante sin cobrar dos veces. El pago que hace el titular desde su celular con el código de la caja pasa exactamente por el mismo camino.

2.3 Tres capas de protección

- **App:** valida para ayudar al usuario, pero no decide.
- **API:** es la autoridad (roles, firma, saldo, PIN, idempotencia).
- **Base de datos:** última barrera: *CHECK* que impide saldos negativos, movimientos de **solo inserción** (sin permisos de modificar y un trigger que lo bloquea) y una vista de **conciliación** (saldo = suma de movimientos).

2.4 Lo que expone la API

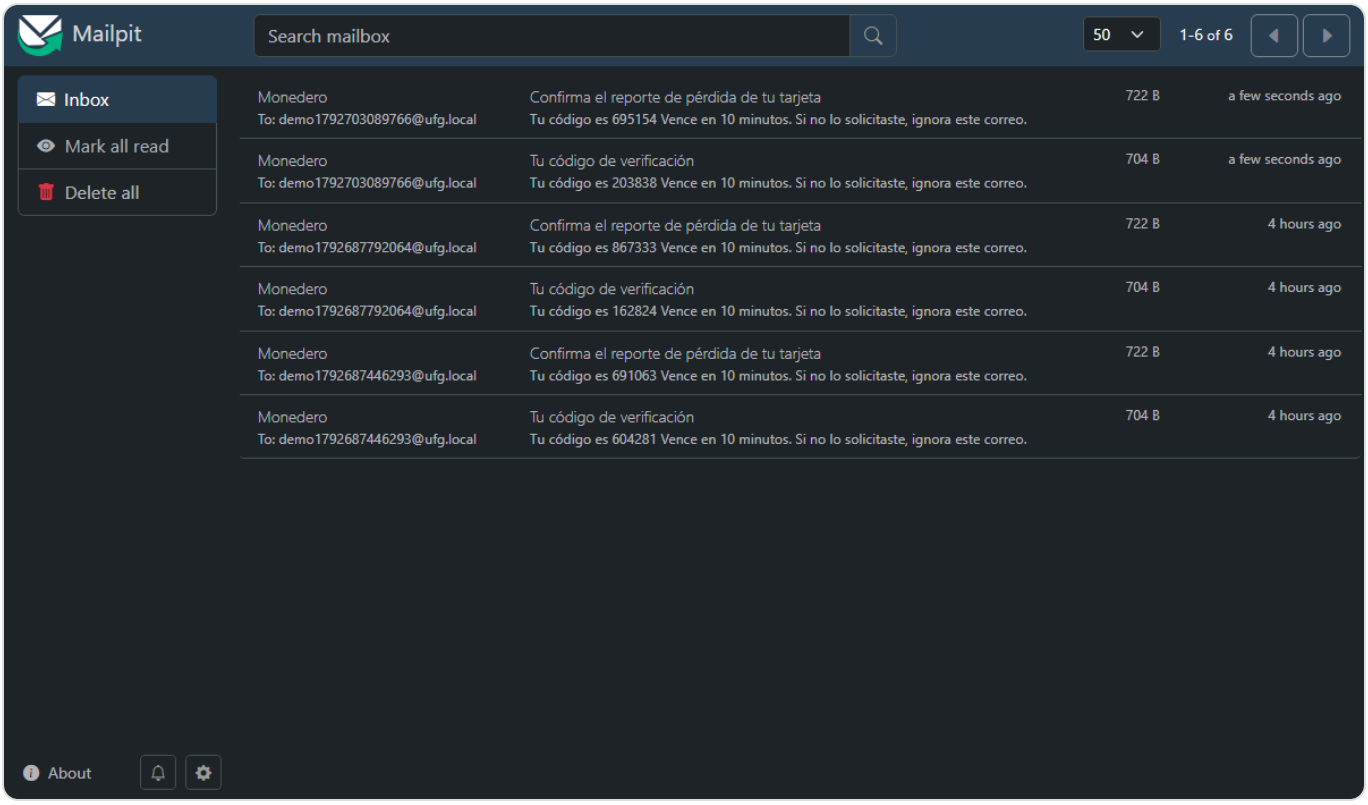
Rol	Operaciones
Público	Registro del titular, verificación del correo con código, inicio de sesión, **renovación y cierre de sesión**
Titular	Mis tarjetas, historial, **pagar en caja con el código de 6 dígitos**, reportar pérdida con código, cambiar PIN
Caja	Leer tarjeta, recargar y cobrar (con credencial de la caja), **su código de caja y sus últimos movimientos**
Administración	**Tablero de indicadores**, emitir y entregar tarjetas, reponer, buscar, conciliación, cajas y **reiniciar la demo**

2.5 Seguridad agregada en esta entrega nuevo

- **Sesión que se renueva sola:** el acceso dura 15 minutos y un token de renovación lo extiende sin pedir la contraseña; cada uso lo cambia por uno nuevo y cerrar sesión lo anula.
- **Límite de solicitudes:** cada cliente tiene un máximo por minuto (más estricto en el inicio de sesión) contra ataques de fuerza bruta.

- **Modo demo controlado:** solo con el modo demo activo, la app muestra en pantalla los códigos que normalmente llegan por correo; en producción se apaga.

Los códigos de verificación y los avisos de cada movimiento se envían por correo a **Mailpit**, un buzón local que no envía nada a Internet:



2.6 Pruebas que demuestran el diseño

59 pruebas automáticas en verde (22 del paquete compartido, 31 de la API y 6 de la app). Las 5 obligatorias corren contra PostgreSQL real:

- **1,000 cobros simultáneos** a la misma tarjeta: 400 aplicados, 600 rechazados por saldo, nunca saldo negativo y conciliación en cero.
- La misma operación enviada **5 veces** se aplica **una sola vez**.
- Una **reposición con falla forzada** a la mitad se revierte completa.
- Intentar **modificar o borrar** un movimiento: la base lo rechaza.
- **Firma inválida** o UID cambiado: la operación se rechaza.

Además hay pruebas de la renovación de sesión, el límite de solicitudes, el modo demo, el tablero, el reinicio de la demo y los flujos de la app (iniciar sesión, cobrar en caja con comprobante, tablero).

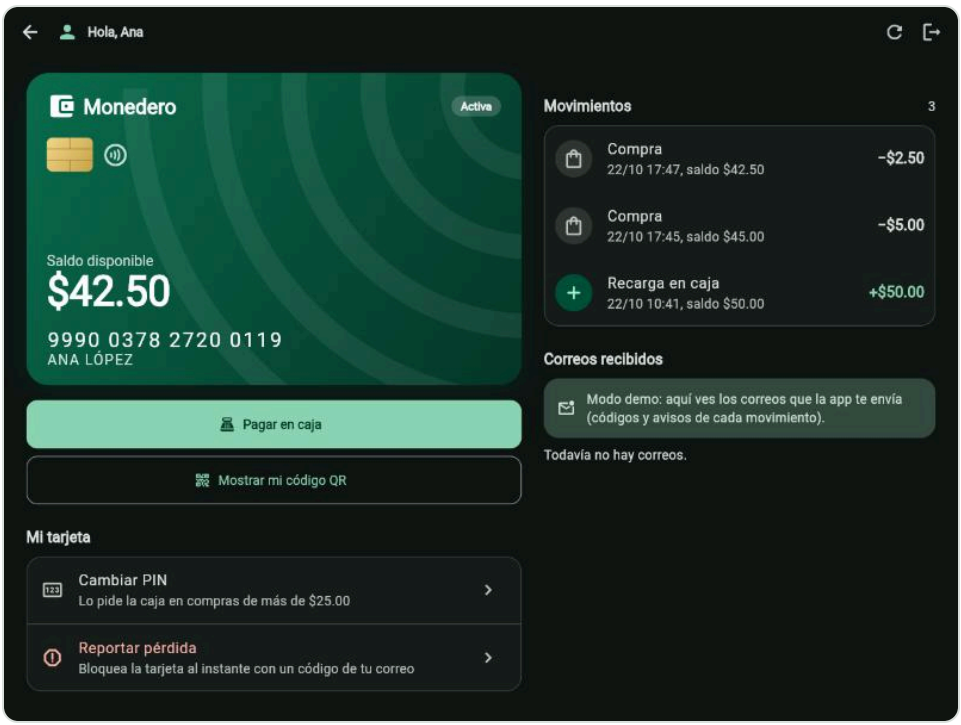
```
== Pruebas: packages/core    +22: All tests passed!
== Pruebas: api              +31: All tests passed!
== Pruebas: apps/app          +6: All tests passed!
  [ok] Sin double/num ni tipos decimales para dinero.
  [ok] Todo en verde.
```

Parte 3 · La app, la demostración y próximos pasos

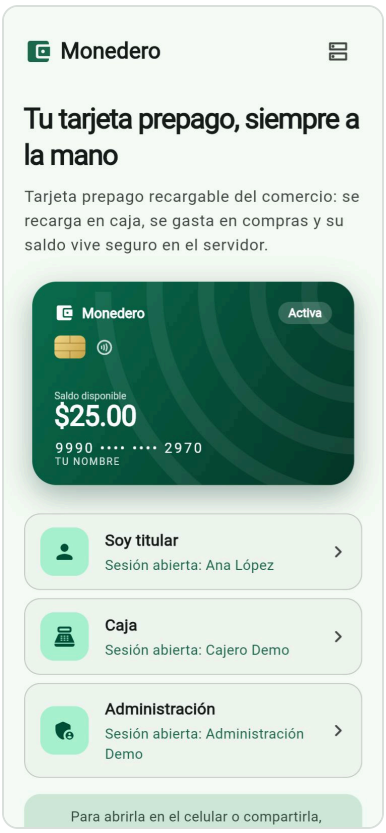
Expositor 3 · ≈ 3 minutos

3.1 La app, ya conectada

Una sola app con tema claro y oscuro que ahora trabaja con **datos reales** de la API, en la web y en el celular. Cada rol inicia sesión con su cuenta; en un mismo navegador pueden convivir las tres pestañas (titular, caja y administración).

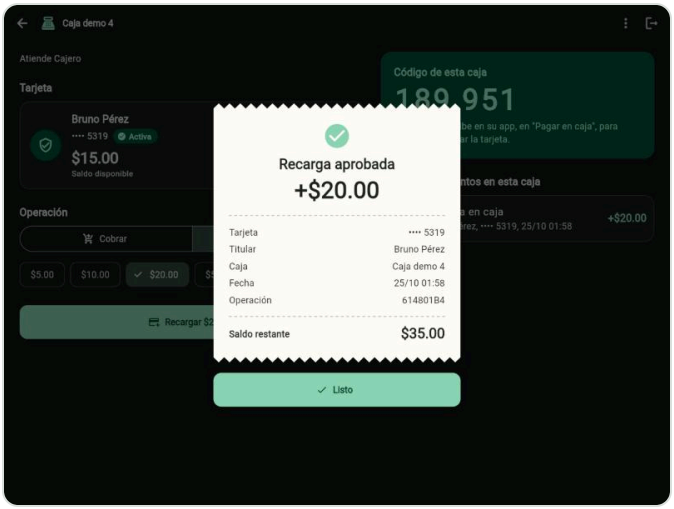


Titular

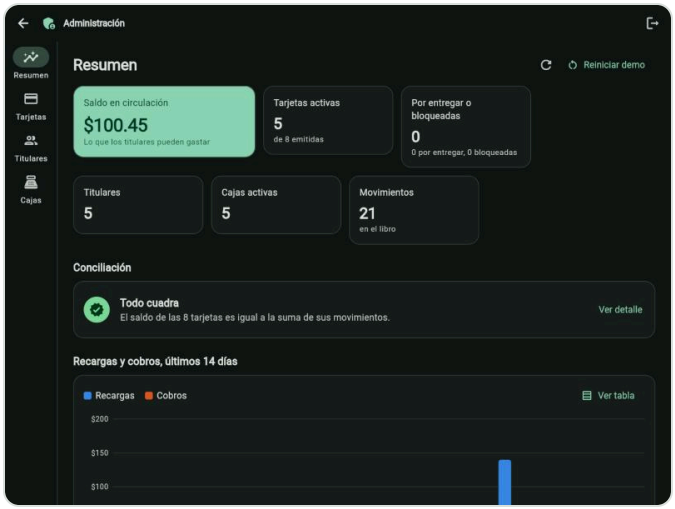


Inicio (celular)

- **Titular:** tarjeta virtual con su saldo, que se actualiza sola; QR para mostrar en caja (número, UID y firma; nunca el saldo); **Pagar en caja** con el código de la caja; movimientos, cambiar PIN y reportar pérdida (la tarjeta se bloquea al instante).
- **Caja:** se vincula una vez con la credencial que da administración; lee la tarjeta con la cámara (QR), de la lista demo o por número; cobra con PIN si pasa de $25, recarga $5, $10, $20, $50 u otro múltiplo y entrega un **comprobante tipo ticket**. Muestra en grande su código para que el titular pague desde su app.
- **Administración:** tablero con el saldo en circulación, la conciliación ("Todo cuadra") y un gráfico de recargas y cobros de 14 días; emitir y entregar, reponer, crear y revocar cajas, y reiniciar la demo.



Caja: comprobante



Administración: tablero

3.2 Demostración

> El día de la exposición se corre `dart run tool/share.dart`: levanta todo e imprime un link `https://….trycloudflare.com` con su QR. El ingeniero lo abre en su computadora o celular (o en la APK) y hace la demo por su cuenta. En modo demo, cada pantalla de inicio de sesión ofrece las cuentas de prueba. Respaldo por consola, contra la misma API: `dart run tool/demo.dart --pausa`.

Paso	Dónde, en la app	Resultado
1. Registrar un titular	Titular → Crear cuenta (el código del correo aparece en pantalla)	Cuenta verificada
2. Emitir la tarjeta	Administración → Titulares → Emitir → Entregar	Tarjeta activa en el celular del titular
3. Recargar $20	Caja → leer tarjeta → Recargar	Ticket y saldo $20.00
4. Cobrar $7.35	Caja → Cobrar, o el titular paga con el código de la caja	Saldo $12.65
5. Sobregiro de $15.00	Caja → Cobrar	"Saldo insuficiente"; saldo intacto
6. Reportar la pérdida	Titular → Reportar pérdida (con código)	Bloqueada; la caja ya no puede cobrar
7. Reponer la tarjeta	Administración → Tarjetas → Reponer	Tarjeta nueva con el mismo saldo
8. Conciliación	Administración → Resumen	"Todo cuadra": diferencia $0.00

Rol	Usuario	Contraseña
Titular	ana@demo.local (PIN 1234)	Titular1234
Caja	cajero@monedero.local	Cajero1234
Administración	admin@monedero.local	Admin1234

3.3 Próximos pasos

Fase	Contenido	Estado
1–2	Estructura, base de datos, API y pruebas obligatorias	**Terminada**
3	Vistas de diseño de los tres roles	**Terminada**
4	Titular y caja conectados a la API: inicio de sesión, saldo real, QR real, lector de tarjetas, cobro, recarga y pago con código	**Terminada**
5	Administración conectada, renovación de sesión y límite de solicitudes	**Terminada**
6	APK de Android firmado con ícono propio, web instalable y link público para la exposición	**Terminada**
7	Tarjetas físicas NFC (MIFARE Classic 1K)	**Siguiente entrega**
8	Servidor en Contabo con HTTPS y respaldos continuos	Requiere aprobación del equipo (tiene costo)

Plan de entregas: esta entrega cierra las fases 1 a 6; la siguiente trae la Fase 7 (NFC); la entrega final incorpora las observaciones del ingeniero y el informe. Queda pendiente elegir el nombre definitivo de la app.